

Reviewer Pack — Subdeterminant Dispersion Lower-Bounds Singular-Tail Matrix ...

Entry 09da8624d6b7 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-27 11:13:06 UTC

Subdeterminant Dispersion Lower-Bounds Singular-Tail Matrix Rigidity

Entry ID: 09da8624d6b7 **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-27 11:13:06 UTC

1. Conjecture

Field A (mathematical branch): Combinatorial discrepancy theory via Chazelle’s subdeterminant lower bound for hereditary discrepancy (Chazelle-Lovász-Vu / Beck-Fiala lineage) **Field B** (complexity object): Matrix rigidity (Valiant) measured by the singular-value tail-rank of $n \times n$ ± 1 sign matrices, the standard pre-Razborov proxy for unsaturated explicit-rigidity bounds

Statement:

For every $n \times n$ ± 1 sign matrix M with $n \geq 4$, define the k -subdeterminant dispersion $\Delta(M) = \max_{2 \leq k \leq 6} \max_{\{S, T \subseteq [n], |S|=|T|=k\}} |\det(M[S, T])|^{\frac{1}{k}} / \sqrt{k}$, and the energy-tail rank $\tau(M) = \#\{i : \sigma_i(M)^2 \geq \|M\|_F^2 / n\}$. Then $\tau(M) \cdot \Delta(M)^2 \geq (1/4) \cdot n$. Equivalently, any matrix whose every small subdeterminant is small (low Chazelle dispersion) must have its energy spread across many singular values.

Rationale (proposer’s reasoning):

Chazelle’s subdeterminant bound is the sharpest combinatorial witness for hereditary discrepancy yet has never been linked directly to singular-tail rigidity, even though both quantify how ‘flat’ a sign matrix can be made by submatrix selection. Bourgain’s $\Lambda(p)$ heuristics suggest small subdeterminants force flat singular spectra (Kashin-style mass-spreading), which would forbid the low-rank-plus-sparse decomposition that defines rigidity collapse. A constant in the bound, if true, gives a poly-time computable explicit-rigidity lower bound that bypasses the natural-proofs barrier (Δ is not a large pseudorandom-closed property: Hadamard matrices saturate it).

Taxonomy category: DISPERSION_DISCREPANCY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): fc276ad7eb9afc74

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across 30 seeds $\times$ 5 ensembles $\times$ 7 sizes $n \in \{6, 8, 10, 12, 14, 16, 20\}$ (1050 matrices), compute $R(M) = \tau(M) \cdot \Delta(M)^2 / n$. Conjecture SUPPORTED iff $\min R \geq 0.25$ over all instances AND

the per-n median R shows no monotone decrease (Spearman $\rho(n, \text{median } R) > -0.5$). FALSIFIED iff any instance yields $R < 0.25$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.92	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.95	SAFE	SAFE
ALGEBRIZATION	SAFE	0.90	SAFE	SAFE
KARP_LIPTON	SAFE	0.90	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - subdeterminant hereditary discrepancy matrix rigidity singular value
- Chazelle determinant lower bound sign matrix singular value tail rank - subdeterminant
dispersion Valiant rigidity Frobenius singular value energy

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
import itertools
from fractions import Fraction

def gaussian_elimination(A):
    n = len(A)
    for i in range(n):
        max_row = i + max(range(i, n), key=lambda r: abs(A[r][i]))
        A[i], A[max_row] = A[max_row], A[i]
        for j in range(i+1, n):
            factor = Fraction(A[j][i], A[i][i])
            for k in range(i, n):
                A[j][k] -= factor * A[i][k]
    return A

def submatrix(matrix, rows, cols):
    return [[matrix[r][c] for c in cols] for r in rows]

def determinant(A):
    if len(A) == 1:
        return A[0][0]
    det = Fraction(0)
    sign = 1
    for j in range(len(A)):
        submat = [row[:j] + row[j+1:] for row in A[1:]]
        det += sign * A[0][j] * determinant(submat)
        sign = -sign
```

```

        det += sign * A[0][j] * determinant(submat)
        sign *= -1
    return det

def subdeterminant_dispersion(M):
    n = len(M)
    k_values = [2, 3]
    max_det_ratio = 0
    for k in k_values:
        for S in itertools.combinations(range(n), k):
            for T in itertools.combinations(range(n), k):
                submat = submatrix(M, S, T)
                det_val = abs(determinant(gaussian_elimination(submat)))
                max_det_ratio = max(max_det_ratio, det_val ** (1/k) / math.sqrt(k))
    return max_det_ratio

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [6, 8, 10, 12, 14, 16, 20]
    results = []

    for n in n_values:
        for _ in range(30):
            M = [[random.choice([-1, 1]) for _ in range(n)] for _ in range(n)]
            delta = subdeterminant_dispersion(M)
            U, s, Vt = map(list, zip(*map(list, zip(*M))))
            tau = sum(s_i**2 >= (sum(x**2 for x in itertools.chain.from_iterable(M)) / n) for s_i in s)
            R = tau * delta**2 / n
            results.append(R)

    min_R = min(results)
    median_R = sorted(results)[len(results) // 2]

    conjecture_holds = min_R >= 0.25 and all(sorted(results)[i] <= sorted(results)[i+1] for i in range(len(results)-1))
    counterexample = "" if conjecture_holds else f"min R={min_R}, median R={median_R}"

    return {
        "metric_name": "R(M)",
        "metric_value": min_R,
        "instances_tested": len(results),
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [11, 23, 37, 53, 71]

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")

    min_Rs = [run_trial(seed)["metric_value"] for seed in seeds]
    median_Rs = sorted(min_Rs)[len(min_Rs) // 2]
    support_fraction = sum(1 for r in min_Rs if r >= 0.25) / len(min_Rs)

    if all(r >= 0.25 for r in min_Rs):

```

```

    print(f"RESULT: SUPPORTED mean={sum(min_Rs)/len(min_Rs)} std={math.sqrt(sum((r - sum(min_Rs)/len(min_Rs))**2)/len(min_Rs))}")
elif any(r < 0.25 for r in min_Rs):
    first_failing_seed = next(i for i, r in enumerate(min_Rs) if r < 0.25)
    print(f"RESULT: FALSIFIED counterexample='min R={min_Rs[first_failing_seed]}', median R={median(min_Rs)}")
else:
    print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_50c2f957.py", line 86, in <module>
    result = run_trial(seed)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_50c2f957.py", line 62, in run_trial
    delta = subdeterminant_dispersion(M)
            ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_50c2f957.py", line 50, in subdeterminant_dispersion
    det_val = abs(determinant(gaussian_elimination(submat)))
                ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_50c2f957.py", line 21, in gaussian_elimination
    A[i], A[max_row] = A[max_row], A[i]
                        ~~~~~
IndexError: list index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed with an IndexError in the gaussian_elimination routine before any R values were produced, so neither the support nor the falsification criterion can be evaluated. | next: Fix the subdeterminant computation (e.g., use numpy.linalg.det or guard the pivot search) and rerun the pre-registered 1050-matrix sweep.

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	62053
2	preregistration	claude_max	opus	0	0	5458
3	novelty	claude_max	opus	0	0	2955

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
4	novelty	claude_max	opus	0	0	6651
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18348
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11460
7	judge	claude_max	opus	0	0	3765

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 110690 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 2}

(full prompt+response transcripts available in research/audit/09da8624d6b7.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/09da8624d6b7.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/09da8624d6b7.tar.gz (if generated)